

Infrastructure as Code (IaC) Provisioning for Finance System using Terraform, Docker and AWS

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no.	Student Name	Enrollment No.
1 .	Krati Patidar	EN22CS301514
2 .	Kamal Yadav	EN22CS301476
3 .	Kashish Sharma	EN22CS301492
4.	Khushi Mehra	EN22CS301504
5.	Jidnyas Bhushan Bhonge	EN22CS301469

Group Name: G03D5

Project Number: DO-03

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

TABLE OF CONTENTS

S.NO .	TOPICS	PAGE NO.
	TITLE PAGE	1
	TABLE OF CONTENTS	2-3
	LIST OF FIGURES	4
1 .	Chapter 1:PROBLEM STATEMENT AND OBJECTIVES	5
	1.1 Problem Statement	5-6
	1.2 Project Objectives	6-7
	1.3 Scope of the Project	7-8
2 .	Chapter 2:PROPOSED SOLUTION	9
	2.1 Key Features	9
	2.2 Overall Architecture / Workflow	10
	2.2.1 System Architecture	10-12
	2.2.2 DFD	12-14
	2.2.3 Use Case	14-15
	2.2.4 Class Diagram	15-16
	2.2.5 Activity Diagram	17-18

	2.2.6 Flow Chart	18-19
	2.2.7 Sequence Diagram	20-21
	2.2.8 Component Diagram	21-22
	2.3 Tools and Technologies Used	22-23
3 .	CHAPTER 3:RESULTS AND OUTPUT	24

	3.1 Screenshots / Outputs	24
	3.1.1 Github Repository Structure of Finance System	24
	3.1.2 AWS EC2 Security Group Inbound Rules Configuration	25
	3.1.3 Docker Image Pull and Container Deployment on AWS EC2	25-26
	3.1.4 Running Docker Container on AWS EC2 Instance	26
	3.1.5 Docker Run Command for Container Deployment	26
	3.1.6 Docker Image Push to Docker Hub	27
	3.1.7 User Interface of Finance System	28
	3.2 Reports / Dashboards / Models	29-30
	3.3 Key Outcomes	30
4.	CHAPTER 4:CONCLUSION	31-32
5 .	CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS	33-34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 2.2.2	DFD	12-14
Figure 2.2.3	Use Case	14-15
Figure 2.2.4	Class Diagram	15-16
Figure 2.2.5	Activity Diagram	17-18
Figure 2.2.6	Flow Chart	18-19
Figure 2.2.7	Sequence Diagram	20-21
Figure 2.2.8	Component Diagram	21-22
Figure 3.1.1	Github Repository Structure of Finance System	24
Figure 3.1.2	AWS EC2 Security Group Inbound Rules Configuration	25
Figure 3.1.3	Docker Image Pull and Container Deployment on AWS EC2	25-26
Figure 3.1.4	Running Docker Container on AWS EC2 Instance	26
Figure 3.1.5	Docker Run Command for Container Deployment	26
Figure 3.1.6	Docker Image Push to Docker Hub	27
Figure 3.1.7	User Interface of Finance System	27

CHAPTER 1: PROBLEM STATEMENT & OBJECTIVES

1.1 PROJECT STATEMENT

In modern application development, infrastructure provisioning is often done manually, which leads to delays, configuration errors, and lack of scalability. For a Finance System, where reliability and consistency are important, manual setup of servers and environments can cause deployment failures and inconsistent performance.

In the traditional approach, developers manually create cloud instances, configure security settings, and install required dependencies. This process is repetitive, time-consuming, and prone to human errors.

Another challenge is the lack of automation in deployment. Without proper automation, updating the application becomes difficult and inconsistent across environments.

To overcome these challenges, there is a need for Infrastructure as Code (IaC), where infrastructure is defined and managed using code. This enables automatic creation of resources and ensures consistency.

In this project, we implemented Infrastructure as Code using Terraform to automate AWS resource provisioning. Docker is used to containerize the application, and GitHub Actions is used to automate deployment.

1.2 PROJECT OBJECTIVES

The objectives of this project are outlined as follows:

The main objectives of this project are:

- To automate infrastructure provisioning using Terraform
- To containerize the Finance System using Docker
- To deploy the application on AWS EC2
- To automate deployment using GitHub Actions
- To reduce manual errors in deployment
- To ensure consistency across environments
- To improve deployment speed and efficiency
- To create a scalable and reusable system

1.3 SCOPE OF THE PROJECT

The scope of this project defines the boundaries and functionalities included in the CI/CD pipeline implementation for the Finance System. The project focuses on automating the software development lifecycle, from code integration to deployment, while ensuring scalability and reliability using cloud infrastructure and containerization.

The scope of the project includes the following:

- 1. Development of CI/CD Pipeline**

Designing and implementing an automated pipeline using GitHub Actions.

- 2. Version Control Integration**

Using Git and GitHub to manage source code and track changes.

- 3. Automated Build and Testing**

Building the application and executing automated test cases.

- 4. Containerization using Docker**

Creating Docker images for consistent application deployment.

- 5. Cloud Deployment on AWS**

Deploying the application on AWS EC2.

- 6. Infrastructure Provisioning using Terraform**

Automating the creation and management of cloud resources.

- 7 . Monitoring and Logging**

Tracking system performance and debugging issues.

- 8 . Basic Security Implementation**

Using authentication and secure credentials for access control.

- 10. Documentation and Reporting**

Providing detailed documentation of system design and implementation.

CHAPTER 2:PROPOSED SOLUTION

2.1 KEY FEATURES

The proposed system provides an automated solution for deploying a Finance System using DevOps tools.

Instead of manually creating infrastructure and deploying applications, Terraform is used to define infrastructure in code. This allows automatic creation of AWS EC2 instances and security groups.

The application is containerized using Docker, which ensures that it runs consistently across different environments.

GitHub Actions is used to automate the deployment process. Whenever code is pushed to GitHub, the pipeline is triggered, which builds and deploys the application automatically. The key features of the system are as follows:

1. Automated CI/CD Pipeline

The system automatically performs build, testing, and deployment whenever code is pushed to the repository.

2. Version Control Integration

Git and GitHub are used to manage source code, track changes, and enable collaboration among developers.

3. Automated Testing

The pipeline executes automated test cases to ensure code quality and prevent faulty deployments.

4. Docker-based Containerization

The application is packaged into Docker containers to ensure consistent execution across environments.

5. Cloud Deployment on AWS

The application is deployed on AWS infrastructure, ensuring scalability and high availability.

6. Infrastructure as Code using Terraform

Terraform is used to automatically provision and manage AWS resources, reducing manual configuration.

7. Scalability and High Availability

The system can automatically scale based on workload and ensures minimal downtime.

8. Monitoring and Logging

The system tracks pipeline execution and application performance for debugging and analysis.

9. Reduced Human Intervention

Automation minimizes manual errors and improves efficiency.

2.2 OVERALL ARCHITECTURE / WORKFLOW

2.1.1 SYSTEM ARCHITECTURE

The system architecture includes the following components:

Developer → writes code

GitHub → stores code

GitHub Actions → automates workflow

Docker → containerizes application

Terraform → provisions infrastructure

AWS EC2 → hosts application

Workflow :

Docker image is built

Terraform creates EC2 instance

EC2 pulls Docker image

Docker container runs

Application is accessed via browser

2.2.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It shows the flow of information between different components such as processes, data stores, and external entities. A DFD helps in understanding how input data is processed and transformed into output within a system. In this project, the DFD represents the flow of data in the CI/CD pipeline integrated with Terraform, Docker, and AWS for deploying the Finance System.

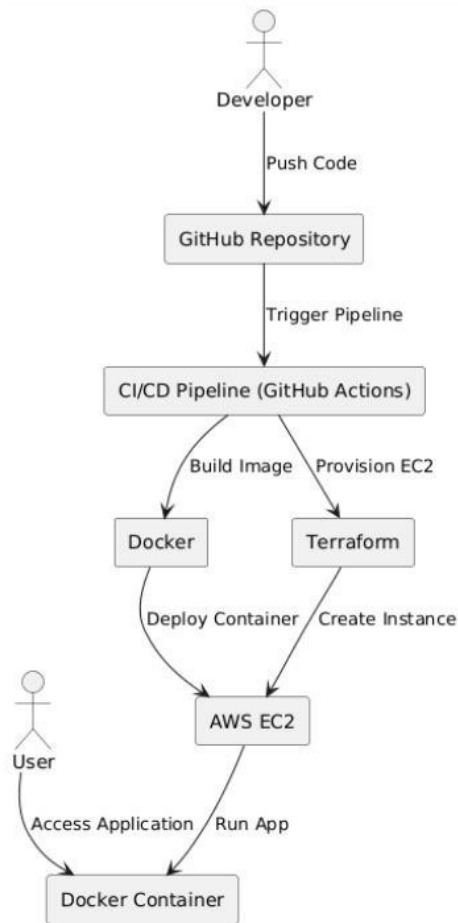


Figure 2.2.2

Working:

1. Developer- The process begins with the developer who writes the application code.

- A. Developer creates or updates the code
- B. Pushes the code to GitHub repository
- C. Initiates the CI/CD workflow

2. GitHub Repository- GitHub acts as the central storage for the application code.

- A. Stores the latest version of the code
- B. Tracks all changes and commits

C. Triggers the CI/CD pipeline automatically

3. CI/CD Pipeline- The CI/CD pipeline automates the build and deployment process.

- A. Pulls the latest code from GitHub
- B. Builds the application
- C. Executes test cases
- D. Initiates Docker image creation
- E. Triggers Terraform scripts

4. Terraform (Infrastructure Layer)- Terraform is used to provision infrastructure on AWS.

- A. Executes Terraform scripts
- B. Creates AWS EC2 instance
- C. Configures security groups (ports access)
- D. Ensures infrastructure is ready for deployment

5. Docker Build System- Docker is used to containerize the application.

- A. Builds Docker image from application code
- B. Packages dependencies and environment
- C. Prepares application for deployment

6. Docker Hub (Registry)- Docker Hub stores the Docker image.

- A. Receives pushed Docker image
- B. Maintains versioned images
- C. Makes image available for deployment

7. AWS EC2 Instance- AWS EC2 acts as the hosting environment.

- A. Pulls Docker image from Docker Hub
- B. Provides cloud infrastructure
- C. Prepares system for running container

8 . Docker Container- The application runs inside a Docker container.

- A. Container is created from Docker image
- B. Ports are mapped
- C. Application starts running

9 . Finance System(Final Output)- This is the final deployed application.

- A. Application is executed successfully
- B. Users can access it via browser

10. User Access- The final step allows access to the application.

- A. User accesses application via public IP

B. Data is processed and results are displayed

2.2.3 USE CASE DIAGRAM

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that represents the interaction between users (actors) and the system. It shows what actions users can perform and how they interact with the system functionalities. In this project, the Use Case Diagram represents how the Developer user interacts with the CI/CD system to build, test, deploy.

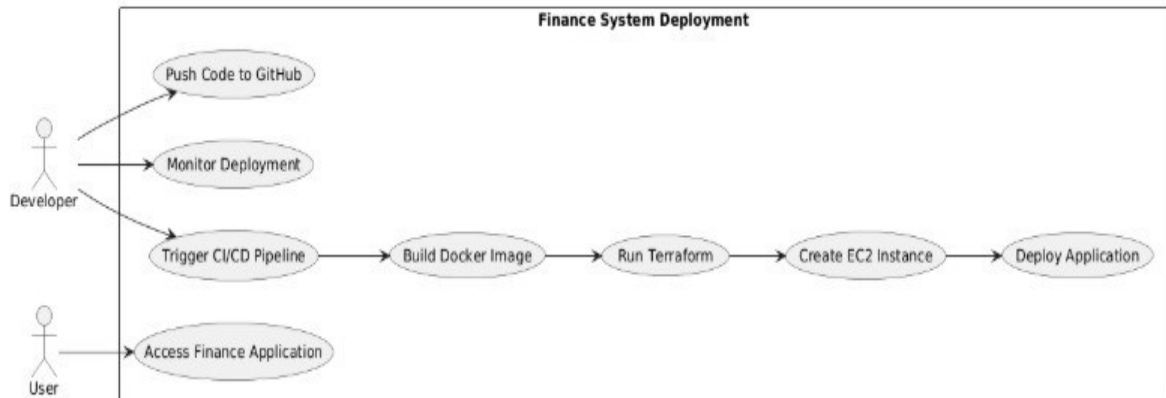


Figure 2.2.3

2.2.5 ACTIVITY DIAGRAM

An Activity Diagram is a UML diagram that represents the flow of activities or steps in a process. It shows how a system performs a sequence of actions from start to end, including decision points and parallel flows. In this project, the activity diagram illustrates the complete workflow of the CI/CD pipeline for deploying the Finance System on AWS EC2.

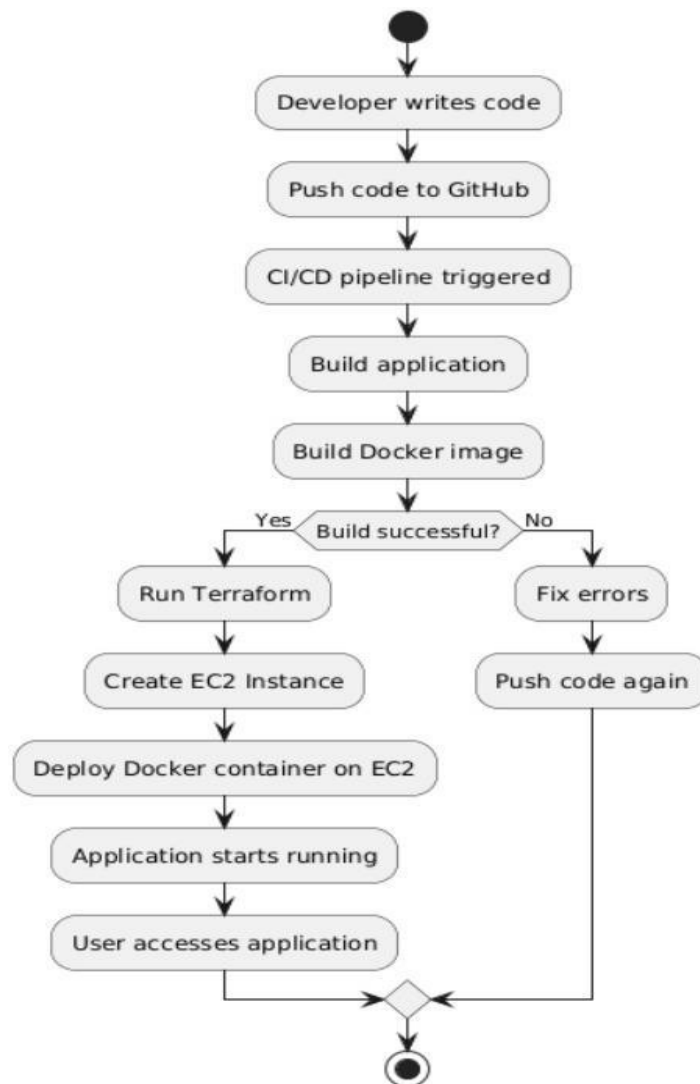


Figure 2.2.5

Working:

- 1.Start of Process-** The process begins with the developer writing the application code.Developer writes code.Prepare application for deployment
- 2.Push Code to Repository-** The developer pushes code to GitHub.Code is uploaded to repository.Version control tracks changes.Pipeline trigger is initiated
- 3.CI/CD Pipeline Triggered-** The pipeline starts automatically after code push.CI/CD tool detects new commit.Pipeline execution begins.Code is pulled from repository

4. Build Application- The application is built. Dependencies are installed (requirements.txt). Application environment is prepared. Build process completes

5. Run Automated Tests- Testing ensures code quality. Test cases are executed. Code correctness is verified. System checks for errors

6. Decision: Tests Passed?- This is the most important decision point.

2.2.6 FLOW CHART

A Flowchart is a graphical representation of a process that shows the sequence of steps using symbols like rectangles (process), diamonds (decision), and arrows (flow). It helps in understanding the logical flow of a system from start to end. In this project, the flowchart represents the complete CI/CD pipeline workflow integrated with Docker, Terraform, and AWS for deploying the Finance System.

Working:

- 1. Start of Process-** The process begins with the developer writing the application code. Developer writes code. Prepares application for deployment
- 2. Push Code to GitHub-** The developer uploads the code to the repository. Code is pushed to GitHub. Version control tracks changes. CI/CD pipeline is triggered automatically
- 3. CI/CD Pipeline Triggered-** The pipeline starts execution. Code is pulled from repository. Pipeline begins build process
- 4. Build Application-** The application is prepared for execution. Dependencies are installed. Application is compiled and configured
- 5. Run Tests-** Testing ensures code quality. Automated test cases are executed. System checks for errors. Ensures application stability
- 6. Decision: Tests Passed?-** This is a decision point in the flowchart.

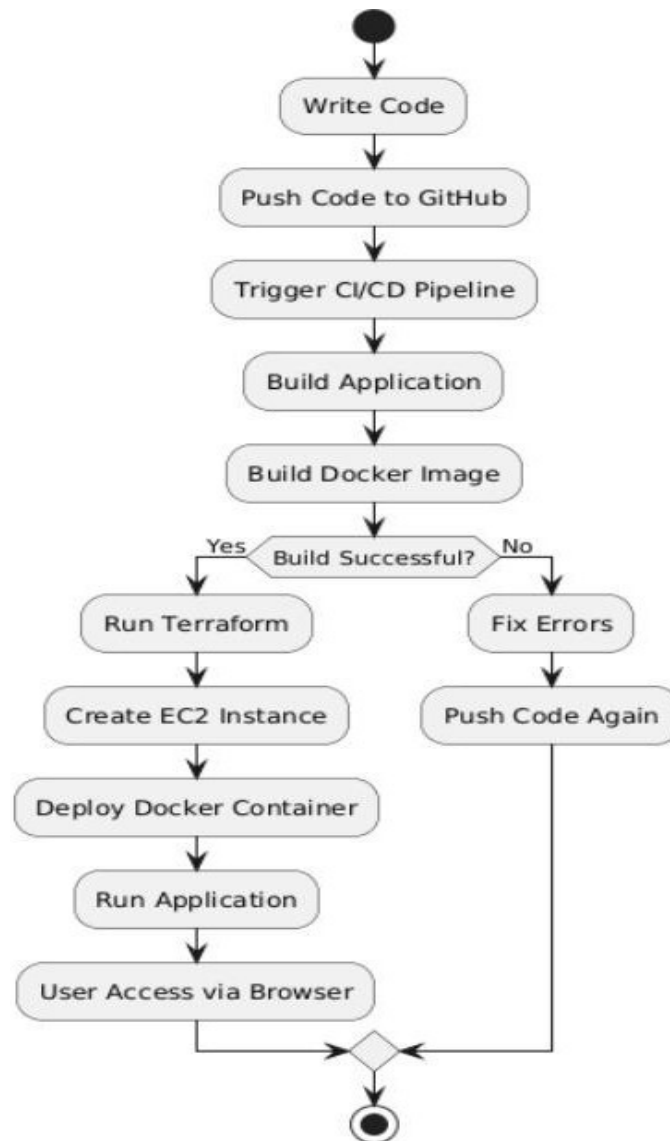


Figure 2.2.6

2.2.7 SEQUENCE DIAGRAM

A Sequence Diagram is a UML diagram that shows how different components of a system interact with each other in a specific sequence over time. It represents the order of messages exchanged between system elements to complete a process. In this project, the sequence diagram illustrates the interaction between Developer, GitHub, CI/CD pipeline, Docker, AWS EC2, and the Finance System during the deployment process.

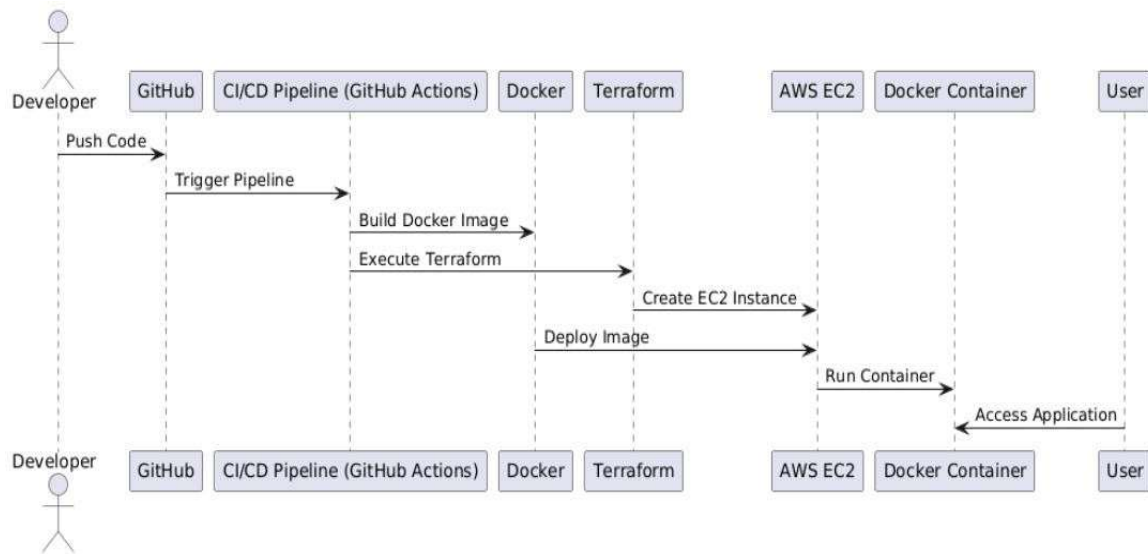


Figure 2.2.7

Working:

1. Components (Participants)- The main components involved in the sequence are:

- A. Developer** – Writes and pushes code
- B. GitHub Repository** – Stores code and triggers pipeline
- C. CI/CD Pipeline (GitHub Actions)** – Automates build and deployment
- D. Docker Engine** – Builds Docker image
- E. Docker Hub** – Stores Docker image
- F. AWS EC2 Instance** – Hosts application
- G. Docker Container** – Runs application
- H. Energy Application** – Final output

2. Step-by-Step Interaction-The sequence of interactions is as follows:

- A. Code Push-** Developer pushes code to GitHub.GitHub receives and stores updated code
- B. Pipeline Trigger-** GitHub triggers CI/CD pipeline.Pipeline starts execution automatically

3. Build and Test- CI/CD pipeline builds the application.Runs automated tests.Verifies code quality

4. Decision: Tests Passed?

2.2.8 COMPONENT DIAGRAM

A Component Diagram is a UML diagram that shows the physical structure of a system by representing its components and how they interact with each other. It focuses on high-level system architecture and how different modules or services are connected. In this project, the component diagram represents the architecture of the CI/CD pipeline for the Finance System, including development, version control, CI/CD system, containerization, infrastructure provisioning, and AWS deployment.

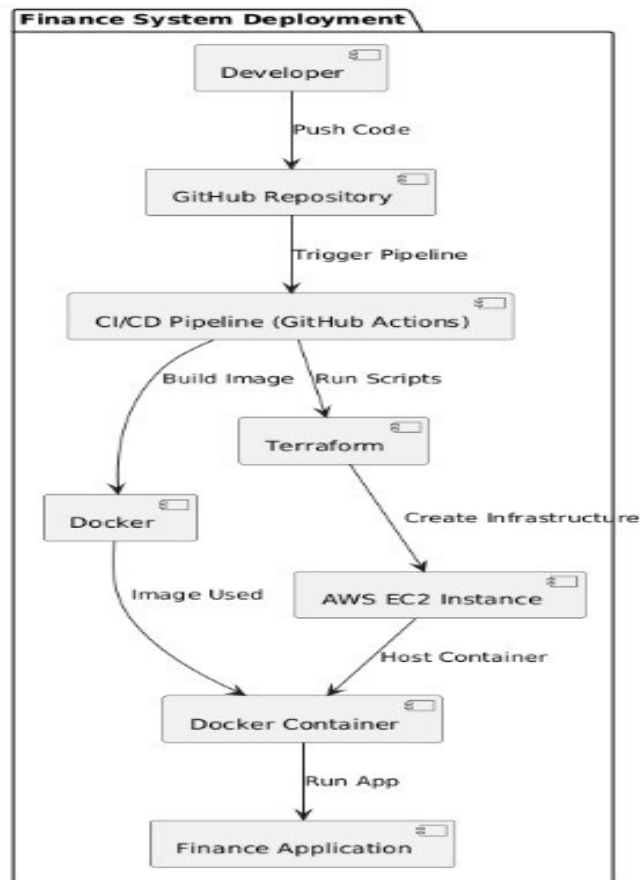


Figure 2.2.8

Working:

1. Developer Side- This is the starting point of the system where development takes place. Developer writes code. Prepares application for deployment. Pushes code to version control system

2. Version Control (GitHub Repository)- This component manages source code and triggers the pipeline.Stores application code.Maintains version history.Triggers CI/CD pipeline on code push
3 . CI/CD System- This is the core component that automates the workflow.

4. Containerization (Docker)- This component handles packaging of the application.Docker Engine builds container image.Docker Image stores application and dependencies.Image is pushed to Docker Hub

5. Infrastructure (Terraform)- This component manages cloud infrastructure..Terraform scripts define AWS resources.Provisions EC2 instance and security groups.Automates infrastructure setup

6. AWS Cloud- This is the deployment environment where the application runs.EC2 instance is created.Docker image is pulled from Docker Hub.Container is executed

7. Docker Container- This component runs the application.Container is created from Docker image.Ports are mapped for access.Application is executed inside container

8. Energy Application- This is the final output of the system.Application starts successfully.Processes user requests.Provides energy insights and reports

2.3 TOOLS AND TECHNOLOGIES USED

1. Frontend

- A. HTML-** Used to create the structure of web pages.
- B. CSS-** Used to style and design the user interface.
- C. JavaScript-** Adds interactivity and dynamic behavior to the web application.

2. Version Control

- A. Git-** A version control system used to track changes in source code.
- B. GitHub-** A platform to store code repositories and collaborate with team members.

3. CI/CD Tools

- A. GitHub Actions-** A CI/CD tool that automates workflows directly from GitHub.

5 . Containerization

- A. Docker-** Used to package the application into containers for consistent deployment.
- B. Dockerfile-** A script containing instructions to build a Docker image.
- C. Docker Hub-** A cloud repository to store and manage Docker images.

6 . Cloud Platform

- A. AWS (Amazon Web Services)-** Provides cloud infrastructure for hosting and deploying applications.
- B. AWS EC2-** Virtual servers used to run applications.

7. Infrastructure as Code

- A. Terraform-** Used to automate the creation and management of cloud infrastructure.

CHAPTER 3:RESULTS AND OUTPUT

The Results and Output section presents the final outcomes of the implemented CI/CD pipeline for the Finance System. This section demonstrates how the system performs after integrating all DevOps tools and deploying the application on AWS using Terraform. It highlights the successful execution of the pipeline, deployment of the application, and the overall improvements achieved through automation.

3.1 SCREENSHOTS / OUTPUTS

3.1.1 GitHub Repository Structure of Energy Application

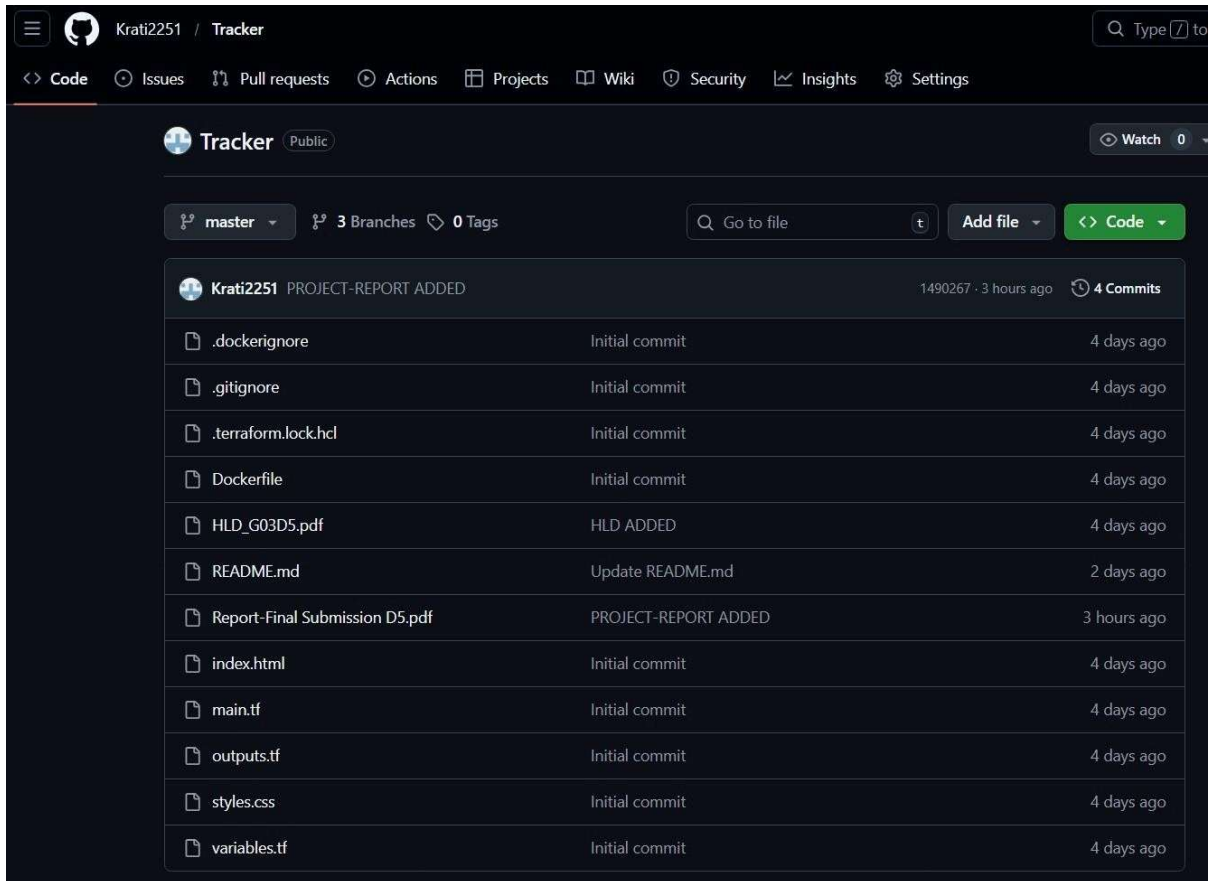


Figure 3.1.1

3.1.2 AWS EC2 Security Group Inbound Rules Configuration

▼ Inbound rules

Filter rules

Security group rule ID	Port range	Protocol	Source	Security groups	Description
sgr-0488b1330de644edf	443	TCP	0.0.0.0/0	launch-wizard-8	-
sgr-003daa4a5f9029142	22	TCP	0.0.0.0/0	launch-wizard-8	-
sgr-0539a449b028ca044	80	TCP	0.0.0.0/0	launch-wizard-8	-
sgr-0c3d32c14703d5539	8501	TCP	0.0.0.0/0	launch-wizard-8	-

Figure 3.1.2

This figure shows the configuration of inbound rules for the AWS EC2 instance used in deploying the Energy Application. The security group allows incoming traffic on essential ports such as HTTP (80), HTTPS (443), and SSH (22), along with custom application ports (8501). All ports are configured with source 0.0.0.0/0, enabling public access to the application. This setup ensures that users can access the deployed application through a web browser and allows secure remote access for management and deployment purposes.

3.1.3 Docker Image Pull and Container Deployment on AWS EC2

This figure shows the process of pulling a Docker image from Docker Hub and deploying it on an AWS EC2 instance. The terminal output displays the successful execution of the docker pull command, followed by listing of Docker images and running the container using the docker run command. It also shows the docker ps output confirming that the container is running successfully with proper port mappings. This demonstrates that the application image has been successfully retrieved, deployed, and executed on the cloud environment.

3.1.4 Running Docker Container on AWS EC2 Instance

This figure shows the output of the docker ps command executed on the AWS EC2 instance, displaying the running Docker container for the Energy Application. It includes details such as container ID, image name, command, creation time, status, and port mappings. The container is running successfully, with ports mapped (8501) to allow external access. This confirms that the application has been successfully deployed and is actively running inside a Docker container on the cloud server.

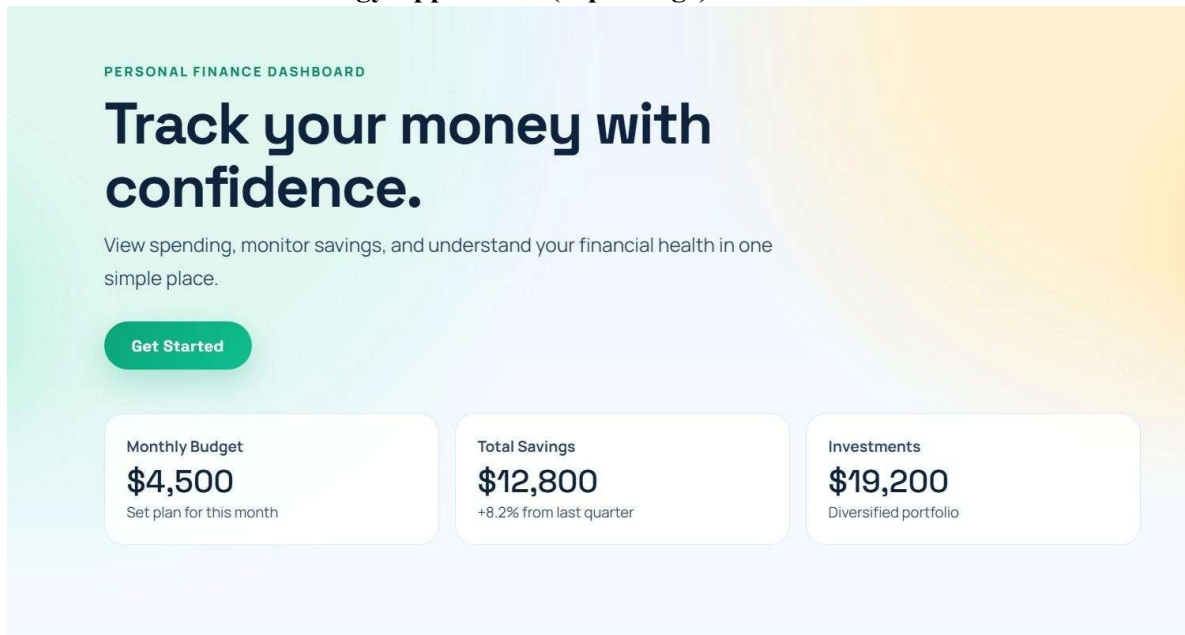
3.1.5 Docker Run Command for Container Deployment

The successful execution confirms that the Docker container has been created and is ready to run the application.

3.1.6 Docker Image Push to Docker Hub

This figure shows the process of pushing a Docker image to Docker Hub using the docker push command. The terminal output indicates that multiple image layers already exist, and only the necessary layers are being uploaded. The progress bar shows the image being successfully pushed to the Docker repository. This confirms that the Docker image is stored in the registry and is ready for deployment on cloud infrastructure.

3.1.7 User Interface of Energy Application (Input Page)



This figure shows the user interface of the deployed Energy Application. It allows users to enter an energy-related query in the input field, such as comparing different energy sources.

The application provides a simple and interactive interface with a “Generate Insight” button to process user input. This confirms that the frontend is successfully integrated with the backend and is accessible through the deployed system

3.2 REPORTS / DASHBOARDS / LOGS

The system provides a dashboard and logs that help in monitoring the deployment process and verifying the successful execution of each stage. These outputs are generated by different tools used in the project, including GitHub Actions, Docker, Terraform, and AWS EC2.

The dashboard and logs provide real-time information about build status, deployment progress, and application availability.

1. CI/CD Dashboard (GitHub Actions)

The CI/CD dashboard in GitHub Actions displays the workflow execution for the project.

Shows pipeline runs triggered after code push

Displays status of each step (success/failure)

Provides detailed logs for build and deployment

Helps in identifying errors and debugging issues

The green status of the pipeline indicates successful deployment of the Finance System.

2. Docker Dashboard and Logs

Docker provides logs and status of containers running on the EC2 instance.

Shows Docker image build process

Displays running containers using `docker ps`

Provides container status and port mapping

Shows logs of application running inside container

These logs confirm that the Finance System is successfully running inside the Docker container.

3. Terraform Dashboard and Logs

Terraform provides execution output when infrastructure is created.

Shows output of `terraform init`, `plan`, and `apply`

Displays creation of EC2 instance

Confirms configuration of security groups

Provides details of resources created

These logs ensure that infrastructure provisioning is automated and completed successfully.

4. AWS EC2 Dashboard

The AWS EC2 dashboard shows the current status of the deployed system.

Displays EC2 instance state (running)

Shows public IP address

Displays security group rules

Confirms instance availability

This dashboard verifies that the application is successfully hosted on AWS.

5. Application Dashboard

The Finance System provides a user interface that acts as the application dashboard.

Displays financial data

Provides structured layout for users

Accessible through browser using public IP

Confirms successful deployment

This dashboard represents the final output of the system.

6. Logs Summary

All logs generated by the system confirm the successful functioning of the deployment pipeline:

CI/CD logs confirm automation

Docker logs confirm container execution

Terraform logs confirm infrastructure creation

AWS dashboard confirms deployment

Application dashboard confirms working system

Conclusion of Dashboard and Logs

The dashboard and logs play a crucial role in monitoring and validating the system. They provide transparency, help in debugging issues, and ensure that the Finance System is deployed successfully and running efficiently.

3.3 KEY OUTCOMES

The implementation of the Finance System using DevOps tools has successfully achieved the objectives of automation, consistency, and efficient deployment. The integration of Terraform, Docker, AWS EC2, and GitHub Actions has resulted in a reliable and scalable deployment system.

1. Successful Infrastructure Provisioning

Terraform was used to automate the creation of AWS infrastructure.

EC2 instance was created automatically

Security group was configured properly

Manual setup was eliminated

Infrastructure became reusable

This confirms the successful implementation of Infrastructure as Code (IaC).

2. Automated Deployment using CI/CD

The CI/CD pipeline using GitHub Actions was successfully implemented.

Pipeline is triggered automatically on code push

Application is built and deployed without manual intervention

Deployment process is faster and efficient

This reduces human errors and ensures consistent deployment.

3. Effective Containerization using Docker

Docker was used to containerize the Finance System.

Docker image was created successfully

Application runs inside container

Same environment maintained across systems

Easy deployment on cloud

This ensures that the application behaves consistently.

4. Successful Cloud Deployment on AWS EC2

The application was deployed on AWS EC2 instance.

EC2 instance is running successfully

Application is accessible via public IP

Security group allows required access

System is available online

This confirms successful hosting of the application.

5. Improved Deployment Efficiency

The overall deployment process has become more efficient.

Reduced manual effort

Faster setup and deployment

Automated workflow

Less chances of configuration errors

6. System Reliability and Consistency

The system provides reliable and consistent results.

Same configuration used every time

No environment mismatch

Stable application performance

7. End-to-End DevOps Pipeline Implementation

The project successfully demonstrates a complete DevOps workflow:

Code → GitHub → CI/CD → Docker → Terraform → AWS EC2 → Application

This shows practical implementation of modern DevOps practices.

CHAPTER 4:CONCLUSION

In this project, we successfully implemented an automated deployment system for the Finance System using modern DevOps tools. The main objective of the project was to automate infrastructure provisioning and application deployment, which has been achieved effectively.

The use of Terraform enabled Infrastructure as Code (IaC), allowing us to define and create AWS resources such as EC2 instances and security groups using code instead of manual configuration. This made the infrastructure setup faster, reusable, and less error-prone.

The application was containerized using Docker, which ensured that the Finance System runs consistently across different environments. Docker helped in packaging the application along with its dependencies, making deployment simple and reliable.

The deployment was carried out on AWS EC2, which provided a scalable and flexible cloud environment for hosting the application. The application is successfully running on EC2 and is accessible through the public IP address.

Additionally, GitHub Actions was used to automate the deployment process through a CI/CD pipeline. This reduced manual intervention and ensured that the application is automatically updated whenever changes are made to the code.

Overall, the project demonstrates how DevOps tools can be integrated to build an efficient, reliable, and automated system. The combination of Terraform, Docker, AWS, and CI/CD has significantly improved deployment speed, reduced manual effort, and ensured consistency.

This project successfully fulfills the goal of implementing Infrastructure as Code along with automated deployment, making the system scalable and suitable for real-world applications.

CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS

Although the project successfully demonstrates Infrastructure as Code (IaC) and automated deployment using DevOps tools, there are several areas where the system can be further improved and extended.

1. Integration with Container Orchestration

In future, the system can be enhanced by integrating container orchestration tools such as Kubernetes. This will help in managing multiple containers, improving scalability, and ensuring high availability of the application.

2. Implementation of Monitoring and Logging Tools

Monitoring tools can be added to track system performance and application health. This will help in identifying issues in real-time and improving system reliability.

3. Enhanced Security Measures

Security can be improved by implementing advanced features such as IAM roles, secure access control, and encrypted communication. This will ensure better protection of data and infrastructure.

4. Multi-Environment Deployment

The system can be extended to support multiple environments such as development, testing, and production. This will improve the testing process and ensure stable releases.

5. Automated Testing Integration

Automated testing can be integrated into the CI/CD pipeline to ensure code quality before deployment. This will reduce bugs and improve application performance.

6. Scalability Improvements

The system can be enhanced to automatically scale resources based on user demand. This will improve performance during high traffic conditions.

7. Improved User Interface

The Finance System can be further enhanced by improving the user interface and adding more features for better user experience